

Variables y colecciones en Python

- Sesión 1 - Introducción a la programación

Docente: José Zevallos



Objetivos de aprendizaje

Al finalizar la sesión, el estudiante:

- comprende qué es una variable y cómo almacenar información en Python;
- distingue tipos básicos como `int`, `float` y `str`;
- reconoce la diferencia entre listas, tuplas y diccionarios;
- resuelve ejercicios simples manipulando datos y colecciones.



Ruta de la sesión

01.

Variables y
tipos básicos

02.

Listas, tuplas
y diccionarios

03.

Ejemplos
guiados en
notebook

04.

Ejercicios
prácticos



01.

</> Variables y tipos de datos



¿Qué es una variable?

Una variable es un nombre que referencia un valor almacenado en memoria.

En Python no se declara el tipo de manera explícita antes de asignar el valor. El tipo se determina automáticamente según el contenido asignado.

Ejemplos:

- ▶ `x = 10`
- ▶ `nombre = "Ana"`
- ▶ `altura = 1.72`

Idea central: una variable permite guardar información para usarla después en operaciones, decisiones o repeticiones.

Tipos básicos en Python

Los tipos básicos que usaremos al inicio son:

- ▶ `int`: números enteros, por ejemplo 5
- ▶ `float`: números decimales, por ejemplo 3.14
- ▶ `str`: texto, por ejemplo "hola"

También es importante revisar el tipo de un valor con:

- ▶ `type(x)`

Esto ayuda a entender por qué algunas operaciones funcionan y otras generan error.

Conversión de tipos

A veces necesitamos transformar un tipo en otro.

- ▶ `int("12")` convierte texto a entero
- ▶ `float("3.14")` convierte texto a decimal
- ▶ `str(100)` convierte número a texto

Esto es útil cuando se reciben datos como texto y se desea operar con ellos numéricamente.

Pregunta para clase:

- ▶ ¿Qué sucede si intentamos sumar "12" y 1 sin conversión?
-

02.

 **Colecciones: listas, tu-
plas y diccionarios**



Listas

Una lista es una colección ordenada y mutable.

Propiedades importantes:

- ▶ mantiene el orden de los elementos;
- ▶ permite modificar, agregar o eliminar elementos;
- ▶ se accede por posición, comenzando en índice 0.

Ejemplos:

- ▶ `nums = [10, 20, 30]`
 - ▶ `nums[0]`
 - ▶ `nums.append(40)`
 - ▶ `nums.pop()`
-

Operaciones frecuentes con listas

En listas son comunes las siguientes operaciones:

- ▶ indexación: `L[0]`
- ▶ slicing: `L[1:]`
- ▶ suma: `sum(L)`
- ▶ mínimo y máximo: `min(L)`, `max(L)`
- ▶ ordenamiento: `sorted(L)` o `L.sort()`

Diferencia importante:

- ▶ `sorted(L)` devuelve una nueva lista;
 - ▶ `L.sort()` modifica la lista original.
-

Tuplas y diccionarios

Tuplas

Una tupla es una colección ordenada e inmutable.

- ▶ ejemplo: `punto = (10, 5)`

Diccionarios

Un diccionario almacena pares clave–valor.

- ▶ ejemplo: `{"nombre": "Pedro", "nota": 15}`
- ▶ acceso: `alumno["nombre"]`
- ▶ modificación: `alumno["nota"] = 18`

Idea práctica: los diccionarios son útiles cuando cada dato tiene una etiqueta.

03.

 **Práctica guiada en notebook**



Secuencia sugerida de trabajo

Durante la práctica se recomienda seguir este orden:

1. ejecutar ejemplos simples de variables;
2. verificar tipos con `type()`;
3. crear y modificar listas;
4. revisar una tupla como ejemplo de estructura inmutable;
5. crear un diccionario con información básica de un estudiante;
6. resolver ejercicios con apoyo del docente.

La meta no es memorizar, sino entender qué estructura conviene según el tipo de información.

Ejercicios propuestos en clase

Ejercicio 1:

- ▶ crear variables `s`, `n` y `f` con texto, entero y decimal.

Ejercicio 2:

- ▶ a partir de `L = [1,2,3]`, agregar 4, crear un slicing y calcular la suma.

Ejercicio 3:

- ▶ construir una tupla de coordenadas.

Ejercicio 4:

- ▶ crear un diccionario con ID, nombre y nota.
-

04.

✓ **Cierre de la sesión**



Ideas de cierre

En esta sesión introducimos las bases para almacenar y organizar información en Python.

Puntos clave:

- ▶ una variable guarda un valor;
- ▶ el tipo del dato importa;
- ▶ una lista permite modificar una colección ordenada;
- ▶ una tupla representa información fija;
- ▶ un diccionario organiza datos mediante claves.

La siguiente sesión se enfocará en tomar decisiones con `if` y repetir instrucciones con bucles.
